

JavaScript Practicals with Answers

[Core JS](#) | [DOM](#) | [Async](#) | [ES6+](#) | [Web Development](#)

Variables, Data Types & Operators · Functions, Arrays & Objects · DOM Manipulation · Events & Forms · Async JS: Promises, Fetch & Async/Await

Practical 1: Variables, Data Types & Operators

Aim: Understand JS variables (var/let/const), data types, and operators.

Code:

```
<script>
// Variable declarations
var oldWay = "avoid var (function-scoped)";
let counter = 10;           // block-scoped, reassignable
const PI    = 3.14159;      // block-scoped, constant

// Data types
let num     = 42;           // Number
let str     = "Hello JS";  // String
let bool    = true;        // Boolean
let arr     = [1, 2, 3];    // Array
let obj     = {name:"Yug", age:20}; // Object
let n       = null;        // Null
let u;                // Undefined

// Type check
console.log(typeof num);    // "number"
console.log(typeof str);    // "string"
console.log(typeof bool);   // "boolean"
console.log(typeof obj);    // "object"

// Operators
console.log(10 + 3);        // 13 (arithmetic)
console.log(10 === "10");   // false (strict equality)
console.log(10 == "10");    // true (loose equality)
console.log(5 > 3 && 2 < 4); // true (logical AND)
console.log(!true);         // false (NOT)

// Template literals
let msg = `Name: ${obj.name}, Age: ${obj.age}`;
console.log(msg);           // "Name: Yug, Age: 20"

// Destructuring
let {name, age} = obj;
let [a, b, c]   = arr;
console.log(name, a);     // "Yug" 1
</script>
```

Explanation:

var: function-scoped, hoisted (avoid). **let:** block-scoped, reassignable. **const:** block-scoped, cannot reassign. **===:** strict (checks type+value). **==:** loose (type coercion). Template literals use backticks and **\${}** for interpolation. Destructuring extracts values from objects/arrays into variables.

Output: Console outputs: 'number', 'string', false (strict), true (loose), template string 'Name: Yug, Age: 20'.

Practical 2: Functions, Arrays & Objects

Aim: Work with functions (declarations, arrow, callbacks) and array/object methods.

Code:

```
<script>
// Function declaration
function greet(name) {
  return `Hello, ${name}!`;
}

// Arrow function
const square = n => n * n;

// Default parameter
const add = (a, b = 10) => a + b;

// Rest parameters
const sum = (...nums) => nums.reduce((acc, n) => acc + n, 0);
```

```

console.log(greet("Yug"));    // "Hello, Yug!"
console.log(square(5));       // 25
console.log(add(5));          // 15 (b defaults to 10)
console.log(sum(1,2,3,4,5)); // 15

// Array Methods
const nums = [1, 2, 3, 4, 5];

const doubled = nums.map(n => n * 2);    // [2,4,6,8,10]
const evens = nums.filter(n => n % 2===0); // [2,4]
const total = nums.reduce((a,n) => a+n,0); // 15
const found = nums.find(n => n > 3);     // 4
const hasEven = nums.some(n => n % 2 === 0); // true

console.log(doubled, evens, total);

// Spread & Rest
const copy = [...nums, 6, 7];
const merged = {...{a:1}, ...{b:2}}; // {a:1, b:2}

// Object methods
const student = { name:"Yug", marks:[85,90,78] };
Object.keys(student);    // ["name","marks"]
Object.values(student);  // ["Yug", [85,90,78]]
</script>

```

Explanation:

Arrow functions: **const fn = (params) => expression**. **map**: transform each element. **filter**: keep matching elements. **reduce**: accumulate to single value. **find**: first match. **some/every**: boolean checks. Spread **...**: copy/merge arrays and objects. **Object.keys/values/entries**: iterate objects.

Output: All console logs confirmed. doubled=[2,4,6,8,10], evens=[2,4], total=15.

Practical 3: DOM Manipulation

Aim: Select, modify, create and delete HTML elements using JavaScript DOM API.

Code:

```

<!DOCTYPE html>
<html>
<head><title>DOM Demo</title></head>
<body>
  <h1 id="title">Hello World</h1>
  <p class="info">Paragraph 1</p>
  <p class="info">Paragraph 2</p>
  <ul id="list"></ul>
  <button id="btn">Change Title</button>

  <script>
    // Selecting elements
    const title = document.getElementById('title');
    const paras = document.querySelectorAll('.info');
    const btn = document.querySelector('#btn');
    const list = document.getElementById('list');

    // Modify content and style
    btn.addEventListener('click', () => {
      title.textContent = 'Title Changed!';
      title.style.color = 'blue';
      title.classList.add('updated');
    });

    // Modify multiple elements
    paras.forEach((p, i) => {
      p.style.backgroundColor = i === 0 ? '#d4a53f' : '#d4c47f';
    });

    // Create & append new elements
  </script>

```

```

const fruits = ['Mango', 'Apple', 'Banana'];
fruits.forEach(fruit => {
  const li = document.createElement('li');
  li.textContent = fruit;
  li.style.cursor = 'pointer';
  li.addEventListener('click', () => li.remove()); // click to delete
  list.appendChild(li);
});

// getAttribute / setAttribute
title.setAttribute('data-lang', 'en');
console.log(title.getAttribute('data-lang')); // "en"
</script>
</body>
</html>

```

Explanation:

getElementById / querySelector / querySelectorAll: select elements. **textContent / innerHTML:** read/write content. **style.property:** inline styles. **classList.add/remove/toggle:** manage CSS classes. **createElement + appendChild:** dynamically add elements. **element.remove():** delete element. **addEventListener('click', fn):** attach events.

Output: Button click changes title to blue. List shows 3 fruits, each disappears on click. Paragraphs get colored backgrounds.

Practical 4: Events & Forms

Aim: Handle DOM events, form input, validation and prevent default behavior.

Code:

```

<!DOCTYPE html>
<html>
<body>
  <form id="regForm">
    <input id="nameIn" type="text" placeholder="Name" required>
    <input id="emailIn" type="email" placeholder="Email" required>
    <button type="submit">Submit</button>
  </form>
  <div id="output"></div>
  <div id="mouse-area"
    style="width:200px;height:100px;background:#dbeafe;
      border-radius:8px;display:flex;align-items:center;
      justify-content:center;margin-top:10px;cursor:pointer">
    Hover / Click Me
  </div>

  <script>
    const form = document.getElementById('regForm');
    const output = document.getElementById('output');
    const area = document.getElementById('mouse-area');

    // Form submit – prevent page reload
    form.addEventListener('submit', e => {
      e.preventDefault();
      const name = document.getElementById('nameIn').value.trim();
      const email = document.getElementById('emailIn').value.trim();
      if (!name || !email) {
        output.innerHTML = '<p style="color:red">All fields required!</p>';
        return;
      }
      output.innerHTML =
        `<p style="color:green">Registered: <b>${name}</b> (${email})</p>`;
      form.reset();
    });

    // Mouse events
    area.addEventListener('mouseenter', () =>
      area.style.background = '#bbf7d0');
    area.addEventListener('mouseleave', () =>
      area.style.background = '#dbeafe');
  </script>

```

```

    area.addEventListener('click', () =>
        area.textContent = 'Clicked! ' + new Date().toLocaleTimeString());

    // Keyboard event
    document.addEventListener('keydown', e =>
        console.log('Key pressed:', e.key));
</script>
</body>
</html>

```

Explanation:

e.preventDefault(): stops default form submission/page reload. **value.trim()**: gets input value without whitespace. **form.reset()**: clears all fields. Event types: **submit**, **click**, **mouseenter**, **mouseleave**, **keydown**, **keyup**, **change**, **input**. **e.key**: which key was pressed. Events bubble up the DOM tree (event propagation).

Output: Form validates and shows green success or red error. Div changes color on hover, shows time on click. Keypresses logged to console.

Practical 5: Async JS: Promises, Fetch & Async/Await

Aim: Handle asynchronous operations using Promises, fetch API, and async/await.

Code:

```

<script>
// Promise
const fetchData = new Promise((resolve, reject) => {
    setTimeout(() => {
        const success = true;
        success ? resolve({name: "Yug", score: 95})
            : reject("Error: Data not found");
    }, 1000);
});

fetchData
    .then(data => console.log("Got:", data.name, data.score))
    .catch(err => console.error(err))
    .finally(() => console.log("Done"));

// Fetch API (GET request)
fetch('https://jsonplaceholder.typicode.com/users/1')
    .then(res => res.json())
    .then(user => console.log("User:", user.name, user.email))
    .catch(err => console.error("Fetch failed:", err));

// Async / Await (cleaner syntax)
async function getUser(id) {
    try {
        const res = await fetch(
            `https://jsonplaceholder.typicode.com/posts/${id}`);
        const data = await res.json();
        console.log("Post:", data.title);
        return data;
    } catch (error) {
        console.error("Error:", error);
    }
}

getUser(1);

// Promise.all – run multiple requests in parallel
async function getMultiple() {
    const [user, post] = await Promise.all([
        fetch('https://jsonplaceholder.typicode.com/users/1').then(r=>r.json()),
        fetch('https://jsonplaceholder.typicode.com/posts/1').then(r=>r.json()),
    ]);
    console.log(user.name, post.title);
}

getMultiple();
</script>

```

Explanation:

Promise: object representing eventual completion/failure. **resolve/reject:** settle promise. **.then/.catch/.finally:** chain handlers. **fetch(url):** returns Promise of HTTP response. **res.json():** parses JSON body. **async function:** function that returns a Promise. **await:** pauses until Promise resolves. **try/catch:** error handling. **Promise.all:** run all in parallel.

Output: After 1s, Promise resolves and logs name/score. Fetch logs user and post data from JSONPlaceholder API.
async/await syntax is cleaner but functionally equivalent to .then chaining.

-- End of Practicals --